

Projeto Integrador Extensionista: Robótica Aplicada

Artigo Técnico

Campo	Conteúdo
Título do projeto	Sistema IoT de Automação e Eficiência Energética para Ambientes Institucionais com Notificação Inteligente via Telegram
Autores (nome, RA)	Gabriel Zanesco Simionato; Anabelle Dors; Bruna Silvera; Gabriel Samudio Cichocki; Henrique Granella Nascimento.
Turma/Período	5º Semestre — Sistemas de Informação — Unimater, Pato Branco/PR

Resumo

Este artigo apresenta o desenvolvimento de um sistema de automação e eficiência energética baseado em Internet das Coisas (IoT), aplicado a ambientes institucionais de ensino. O problema central consiste no elevado desperdício energético decorrente do esquecimento de iluminação em salas de aula desocupadas, situação recorrente em instituições de ensino superior como a Unimater, localizada em Pato Branco/PR.

A solução proposta integra um sensor de movimento (PIR HC-SR501) e um sensor de luminosidade (LDR) ao microcontrolador ESP32, que atua na camada de borda, responsável pelo sensoriamento, pela publicação dos eventos via MQTT e pelo acionamento do relé de iluminação, seja por comando remoto, seja pelo interruptor físico. O ESP32 conecta-se diretamente à rede Wi-Fi e publica os eventos de ocupação e luminosidade, via protocolo MQTT, em um broker centralizado. O back-end, uma aplicação FastAPI hospedada em nuvem (Railway), subscreve esses eventos, mantém o estado atual em Redis, registra o histórico em InfluxDB e aciona a Telegram Bot API para disparo das notificações ao coordenador, com opções de ação remota por botões inline (“Manter Ligado” / “Desligar”); os mesmos eventos alimentam um painel web (Next.js) em tempo real por meio de SSE. Na ausência de resposta dentro do período configurado, o coordenador é alertado de forma recorrente. O sistema oferece ainda controle manual integrado por meio de uma chave física (DPDT), tratado localmente pelo ESP32 e operante mesmo sem o back-end; o controle autônomo baseado nos sensores (PIR/LDR) permanece como trabalho futuro.

O estudo demonstra viabilidade técnica e econômica: considerando o desperdício energético típico de uma sala de aula, estima-se um potencial de economia anual de até R\$ 1.320,00 por ambiente, com baixo custo de implantação e payback reduzido. O sistema alinha-se aos Objetivos de Desenvolvimento Sustentável ODS 7 (Energia Limpa), ODS 11 (Cidades Sustentáveis) e ODS 13 (Ação Climática), representando uma contribuição concreta para a digitalização da infraestrutura predial em contextos de baixo orçamento.

Palavras-chave: *IoT, automação predial, eficiência energética, ESP32, MQTT.*

Abstract

This article presents the development of an automation and energy efficiency system based on the Internet of Things (IoT), applied to institutional educational environments. The central problem is the significant energy waste caused by lighting being left on in unoccupied classrooms, a recurring situation in higher education institutions such as Unimater, located in Pato Branco, Paraná, Brazil.

The proposed solution integrates a motion sensor (PIR HC-SR501) and a light sensor (LDR) with the ESP32 microcontroller, which operates at the edge layer, responsible for sensing, publishing events via MQTT, and actuating the lighting relay, either by remote command or by the physical switch. The ESP32 connects directly to the Wi-Fi network and publishes occupancy and luminosity events, via the MQTT protocol, to a centralized broker. The back-end, a FastAPI application hosted in the cloud (Railway), subscribes to these events, keeps the current state in Redis, stores the history in InfluxDB, and triggers the Telegram Bot API to dispatch notifications to the coordinator, with remote action options through inline buttons (“Keep on” / “Turn off”); the same events feed a real-time web dashboard (Next.js) through SSE. In the absence of a response within the configured period, the coordinator is repeatedly alerted. The system also provides integrated manual control through a physical switch (DPDT), handled locally by the ESP32 and operational even without the back-end; autonomous sensor-based (PIR/LDR) control remains future work.

The study demonstrates technical and economic feasibility: considering the typical energy waste of a classroom, an annual savings potential of up to BRL 1,320.00 per environment is estimated, with low implementation cost and a short payback period. The system aligns with the Sustainable Development Goals SDG 7 (Affordable

and Clean Energy), SDG 11 (Sustainable Cities and Communities), and SDG 13 (Climate Action), representing a concrete contribution to the digitalization of building infrastructure in low-budget contexts.

Keywords: *IoT, building automation, energy efficiency, ESP32, MQTT.*

1. Introdução

O consumo energético em edificações representa parcela significativa da demanda elétrica nacional. No segmento educacional, a ausência de automação predial resulta em desperdício sistemático de energia elétrica: iluminação e sistemas de climatização permanecem em operação em salas de aula desocupadas por períodos indeterminados, seja por esquecimento dos usuários ou pela ausência de mecanismos de controle automático.

No contexto da Unimater, Centro Universitário localizado em Pato Branco, Paraná, esse fenômeno é observável cotidianamente. Uma sala de aula típica com capacidade para 30 alunos consome aproximadamente 2.200 W entre iluminação (10 luminárias LED de 40 W) e ar-condicionado split de 18.000 BTU (~1.800 W). Considerando que esse conjunto permaneça ligado indevidamente por quatro horas fora do horário de uso em dias letivos, o desperdício chega a 8,8 kWh por dia, ou aproximadamente R\$ 1.320,00 por sala por ano, aplicando a tarifa média da distribuidora regional COPEL de R\$ 0,75/kWh.

A presente proposta endereça este problema por meio de um sistema IoT de baixo custo, baseado no microcontrolador ESP32-C6 da Espressif Systems, operando como nó autônomo com conectividade Wi-Fi na camada de borda. Nesta etapa, o sistema concentra-se na automação da iluminação. O diferencial arquitetural reside na separação clara de responsabilidades: o ESP32 concentra-se no sensoramento, na atuação local e na publicação dos eventos via MQTT, enquanto a orquestração, o armazenamento de estado e a comunicação com o coordenador são processados por um back-end em nuvem. Essa divisão garante clareza arquitetural: o nó de borda permanece simples e desacoplado da lógica de negócio, que é centralizada no back-end. O controle autônomo baseado nos sensores (PIR/LDR) não é implementado nesta etapa; já o acionamento local pela chave física (DPDT) opera de forma independente do back-end, garantindo o controle manual da iluminação mesmo durante eventual indisponibilidade da nuvem.

1.1 Justificativa e alinhamento com ODS/ESG

O projeto apresenta aderência direta a três Objetivos de Desenvolvimento Sustentável da Agenda 2030 da ONU, conforme demonstrado na Tabela 1.

Tabela 1 — Alinhamento do projeto com os Objetivos de Desenvolvimento Sustentável (ODS) da Agenda 2030 da ONU.

ODS (número e nome)	Justificativa local	Indicador / Medida de Impacto
ODS 7 — Energia Limpa e Acessível	Redução do desperdício energético em salas de aula da Unimater por meio de automação com desligamento inteligente da iluminação.	Redução do consumo indevido por sala; economia anual estimada de até ~R\$ 1.320,00 por ambiente (potencial total da sala).
ODS 11 — Cidades e Comunidades Sustentáveis	Digitalização da infraestrutura predial de instituições de ensino no município de Pato Branco, promovendo building automation de baixo custo.	Custo de implantação reduzido por sala, com payback estimado em poucos meses (detalhamento na BOM).
ODS 13 — Ação Contra a Mudança Global do Clima	Redução indireta de emissões de CO ₂ associadas ao consumo elétrico desnecessário em períodos de sala vazia.	Diminuição proporcional da pegada de carbono do campus com base na matriz energética regional.

Fonte: Elaborado pelos autores (2026).

Do ponto de vista ESG (Environmental, Social and Governance), a iniciativa contribui para a redução da pegada de carbono institucional, melhoria da eficiência operacional e modernização da gestão predial, indicadores crescentemente demandados por órgãos financiadores e processos de acreditação institucional.

1.2 Objetivos

O objetivo geral deste trabalho consiste em desenvolver e validar um sistema IoT de baixo custo para automação do gerenciamento da iluminação de salas de aula, com notificação inteligente ao coordenador e capacidade de atuação remota.

Para alcançar esse objetivo, foram estabelecidos os seguintes objetivos específicos: implementar a lógica de detecção de ausência de ocupação com período de alerta configurável pelo gestor; integrar notificação via Telegram com opções de ação remota por botões inline (“Manter Ligado” / “Desligar”); acionar relé de estado sólido para controle da iluminação da sala mediante comando; estruturar o nó de borda de forma desacoplada, publicando os eventos dos sensores e executando os comandos recebidos; e quantificar o impacto energético e econômico do sistema em ambiente real.

1.3 Escopo e limitações iniciais

O MVP (Minimum Viable Product) desta fase contempla a implantação e validação em uma única sala de aula, com um nó ESP32, dois sensores (PIR + LDR), um relé SSR para a iluminação, back-end em nuvem (Railway) e painel web (Next.js/Vercel) para visualização em tempo real, além da interface de notificação via Telegram. Ficam fora do escopo desta etapa: a automação da climatização (ar-condicionado), a integração com sistema de gestão acadêmica (ERP), o controle de múltiplas salas em larga escala e o módulo de predição de ocupação por Machine Learning, previstos como trabalhos futuros.

2. Pesquisa e fundamentação

O estado da arte em Building Automation Systems (BAS) é dominado por soluções comerciais de alto custo, como a plataforma EcoStruxure da Schneider Electric e o Desigo CC da Siemens, cujos custos de implantação inviabilizam a adoção por instituições de pequeno e médio porte. O presente projeto posiciona-se na intersecção entre IoT de baixo custo e automação predial, segmento em crescimento exponencial impulsionado pela redução de custo dos microcontroladores e pela proliferação de APIs de comunicação abertas.

O protocolo MQTT (Message Queuing Telemetry Transport), padronizado pela OASIS sob a especificação v5.0, é amplamente adotado em aplicações IoT pela sua leveza, modelo publish/subscribe e suporte a Quality of Service (QoS) configurável, características essenciais para ambientes com conectividade variável. O broker Mosquitto, mantido pela Eclipse Foundation, constitui implementação de referência open-source para ambientes de pequena e média escala.

O ESP32-C6, da Espressif Systems, é um microcontrolador de baixo custo baseado em arquitetura RISC-V (single-core, até 160 MHz), com conectividade Wi-Fi 6 (802.11ax) e Bluetooth LE 5 integrados, amplamente adotado em aplicações IoT. No contexto deste projeto, o ESP32-C6 (módulo DevKit) é utilizado como nó de borda autônomo, conectando-se diretamente à rede Wi-Fi e publicando os eventos de ocupação e luminosidade no broker MQTT via TCP/IP. O suporte oficial via ESP-IDF SDK v5.x torna este componente particularmente adequado para desenvolvimento acadêmico e prototipagem industrial.

2.1 Referências técnicas e projetos semelhantes

As principais referências técnicas consultadas, bem como o conhecimento aproveitado de cada uma e sua aplicação no projeto, são apresentadas na Tabela 2.

Tabela 2 — Referências técnicas consultadas e sua aplicação no projeto.

Fonte	O que foi aprendido/aproveitado	Aplicação no projeto
ESP-IDF / Espressif Systems — ESP32-C6 Technical Reference Manual (2023)	ESP32-C6 (RISC-V) utilizado como nó de borda com conectividade Wi-Fi; modos de economia de energia (deep sleep, modem sleep); comunicação direta com o broker MQTT via TCP/IP.	Base para seleção do microcontrolador e definição dos modos de economia de energia embarcados.
OASIS — MQTT Version 5.0 Specification (2019)	Protocolo publish/subscribe leve, QoS 0/1/2, adequado para IoT com baixa largura de banda.	Protocolo de comunicação entre o ESP32 e o broker Mosquitto hospedado em nuvem.
Módulo sensor de luminosidade (LDR)	Sensor de luminosidade de baixo custo com saída para GPIO; permite distinguir condições de claro/escuro por comparação a um limiar.	Medição da luminosidade ambiente para correlação com o estado da iluminação artificial e redução de falsos acionamentos.
Telegram — Bot API (core.telegram.org/bots/api, 2024)	API oficial e gratuita para criação de bots no Telegram; suporte a mensagens, botões inline (InlineKeyboardMarkup) e webhooks; sem necessidade de middleware ou aprovação de conta comercial.	Camada de notificação para alertas ao coordenador com botões inline de ação remota (“Manter Ligado” / “Desligar”), sem custo de licença e sem dependência de middleware externo.
FastAPI — Documentação Oficial (fastapi.tiangolo.com, 2024)	Framework web Python assíncrono de alto desempenho; integração com MQTT via aiomqtt e suporte nativo a Server-Sent Events (SSE) para envio de eventos em tempo real ao navegador.	Back-end central (hospedado na Railway) que subscreve os tópicos MQTT, orquestra as regras de negócio e propaga os eventos ao painel web via SSE.
Redis — Documentação Oficial (redis.io, 2024)	Banco de dados em memória de baixa latência; armazenamento de estado por chave (sala:{id}) e do parâmetro de timeout configurável em tempo de execução.	Camada de estado atual das salas, permitindo respostas imediatas ao painel e alteração do timeout via Telegram sem reimplantação.

Fonte: Elaborado pelos autores (2026).

3. Ideia

Esta seção apresenta a fundamentação da proposta: o problema central identificado no ambiente institucional, a solução de automação concebida para endereçá-lo e os requisitos funcionais e não funcionais que orientaram o desenvolvimento.

3.1 Descrição do problema

Instituições de ensino superior operam suas instalações em múltiplos turnos diários, com salas de aula que alternam entre períodos de alta ocupação e desocupação completa. A transição entre esses estados frequentemente não é acompanhada pelo desligamento adequado dos sistemas elétricos.

A ausência de automação obriga a instituição a depender exclusivamente da ação humana para o gerenciamento energético. Professores, técnicos e alunos frequentemente esquecem de desligar a iluminação ao deixar as salas, comportamento amplamente documentado em auditorias energéticas de edificações públicas e educacionais. Esse problema afeta diretamente a instituição, em termos de custo financeiro, o meio ambiente, por meio das emissões de carbono geradas, e indiretamente os alunos, cuja qualidade do ambiente de aprendizagem é comprometida. O desperdício estimado é de 8,8 kWh/sala/dia, equivalente a R\$ 1.320,00/sala/ano. Em uma instituição com 20 salas, o impacto financeiro supera R\$ 26.000,00 anuais, recursos que poderiam ser redirecionados para investimentos pedagógicos.

3.2 Descrição da automação proposta

O sistema opera segundo o seguinte fluxo lógico: como entrada, o sensor PIR HC-SR501 monitora continuamente a presença de movimento na sala, enquanto o módulo LDR informa a condição de luminosidade do ambiente. No processamento, o ESP32 executa a leitura periódica dos sensores e publica os eventos de ocupação e luminosidade via MQTT, sem aplicar lógica autônoma baseada nesses sensores. O back-end FastAPI, hospedado em nuvem (Railway), subscreve esses tópicos, atualiza o estado atual da sala no Redis, registra o histórico no InfluxDB e propaga os eventos ao painel web em tempo real por meio de SSE. O serviço de Telegram verifica periodicamente se alguma sala permanece vazia com a iluminação acesa além do período configurado e, em caso afirmativo, envia ao coordenador uma notificação com botões inline. Como saída, o sistema apresenta o estado das salas no painel web e no Telegram e aciona o relé SSR para controle da iluminação, seja por comando do back-end, em resposta à decisão do coordenador, seja pelo acionamento direto do interruptor físico (chave DPDT), tratado localmente pelo ESP32.

O ganho esperado é duplo: financeiro, pela redução do desperdício energético, e operacional, pela liberação do coordenador de rondas físicas periódicas para verificação manual das salas.

3.3 Requisitos e critérios de sucesso

Os requisitos funcionais e não funcionais do sistema, bem como os critérios de validação e prioridades associadas, são apresentados na Tabela 3.

Tabela 3 — Requisitos funcionais e não funcionais do sistema com critérios de validação.

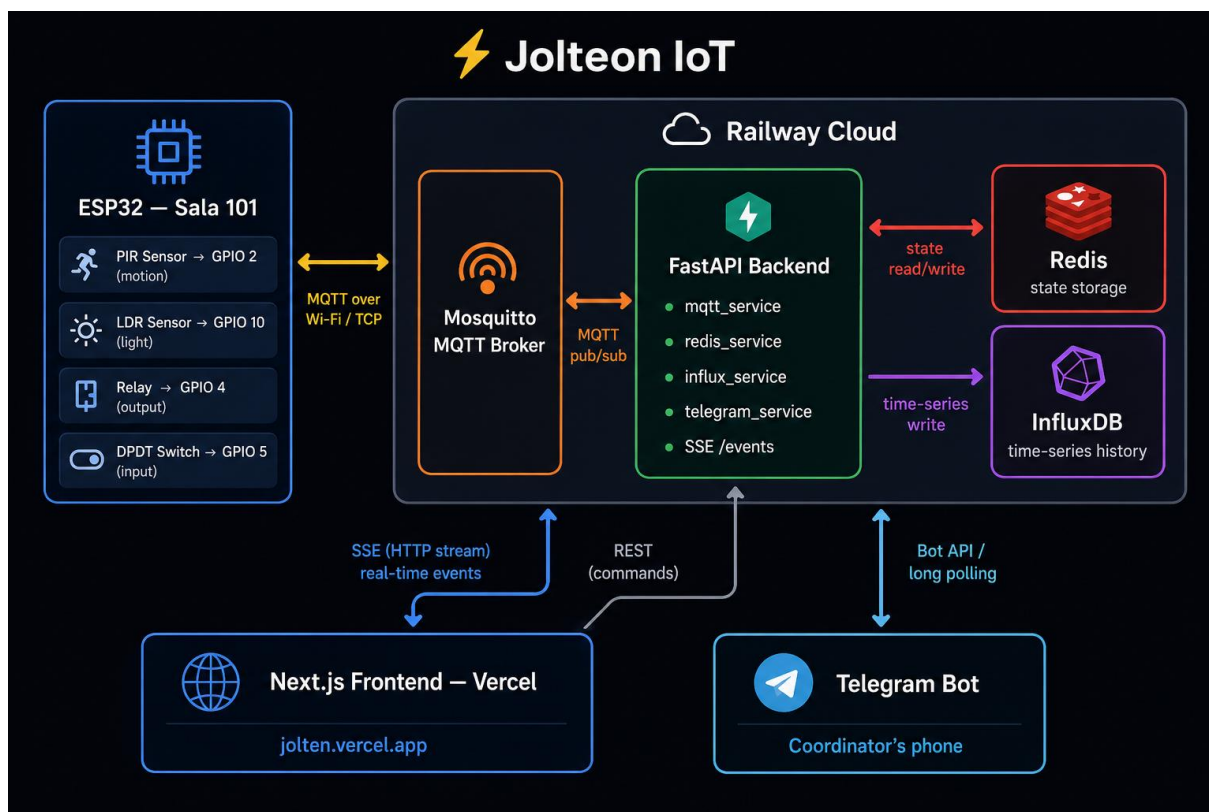
Requisito (funcional ou não funcional)	Como medir/validar	Prioridade
RF01 — Detectar ausência de movimento pelo período configurado pelo coordenador	Simular ausência por N minutos e verificar disparo da notificação	Alta
RF02 — Enviar notificação via Telegram ao coordenador com opções de ação	Verificar recebimento da mensagem com botões inline “Manter Ligado” / “Desligar” em até 30 segundos	Alta
RF03 — Desligar a iluminação remotamente via botão inline no Telegram	Acionar relé SSR e confirmar estado OFF via MQTT	Alta
RF04 — Reenviar o alerta ao coordenador enquanto a sala permanecer vazia com a iluminação acesa	Simular ausência de resposta e verificar o reenvio do alerta após o período configurado	Alta
RF05 — Permitir ajuste do tempo de ausência pelo coordenador via interface web	Alterar parâmetro na interface e verificar comportamento atualizado do sistema	Média
RNF01 — Tempo de detecção a disparo da notificação inferior a 60 segundos	Medir latência com cronômetro em 10 ciclos de teste	Alta
RNF02 — Disponibilidade do sistema acima de 99% em horário letivo	Monitorar uptime via painel web (dashboard) durante o período de validação	Média
RNF03 — Não coletar dados pessoais identificáveis (conformidade LGPD)	Auditoria do tráfego MQTT — apenas estado binário (presença/ausência) deve ser transmitido	Alta

Fonte: Elaborado pelos autores (2026).

4. Arquitetura da solução

A arquitetura do sistema é estruturada de forma a garantir a eficiência na coleta de dados e a autonomia na execução das ações de controle. A Figura 1 apresenta o diagrama de blocos da arquitetura completa do sistema, organizado nas três camadas funcionais que serão detalhadas a seguir.

Figura 1 — Arquitetura do sistema Jolteon IoT: nó de borda (ESP32), back-end em nuvem (Railway) e frontend web (Vercel), com notificação via Telegram

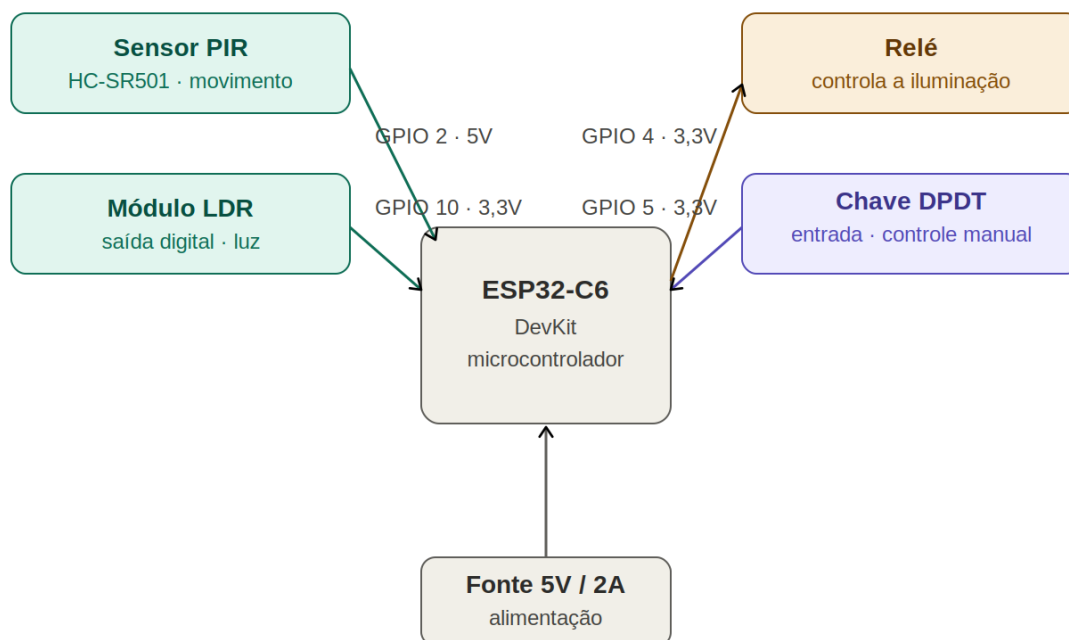


Fonte: Elaborado pelos autores (2026).

4.1 Diagrama/fluxo da solução

A arquitetura é composta por três camadas funcionais. A camada de Percepção (Edge) é formada pelo ESP32 conectado ao sensor PIR HC-SR501 (GPIO 2) e ao módulo LDR (GPIO 10), acionando o relé de iluminação pelo GPIO 4. Integra ainda uma chave física DPDT, lida pelo GPIO 5, que implementa o controle manual da iluminação de forma integrada ao controle remoto (detalhado na Seção 7.1). O microcontrolador conecta-se diretamente à rede Wi-Fi institucional, realiza a leitura periódica dos sensores e publica os eventos de ocupação, luminosidade e estado do interruptor via MQTT. O acionamento do relé ocorre por comando do back-end ou, localmente, pelo toque na chave DPDT; o nó de borda não aplica, contudo, lógica autônoma baseada nos sensores PIR/LDR, que permanece como trabalho futuro. O mapeamento das conexões de hardware do nó de borda é apresentado na Figura 2.

Figura 2 — Diagrama de conexões de hardware do nó de borda: ESP32-C6 DevKit com sensor PIR (GPIO 2), módulo LDR (GPIO 10), relé de iluminação (GPIO 4) e chave DPDT de controle manual (GPIO 5).



PIR e LDR são entradas; o relé é saída de acionamento; a chave DPDT é a entrada de controle manual.

Fonte: Elaborado pelos autores (2026).

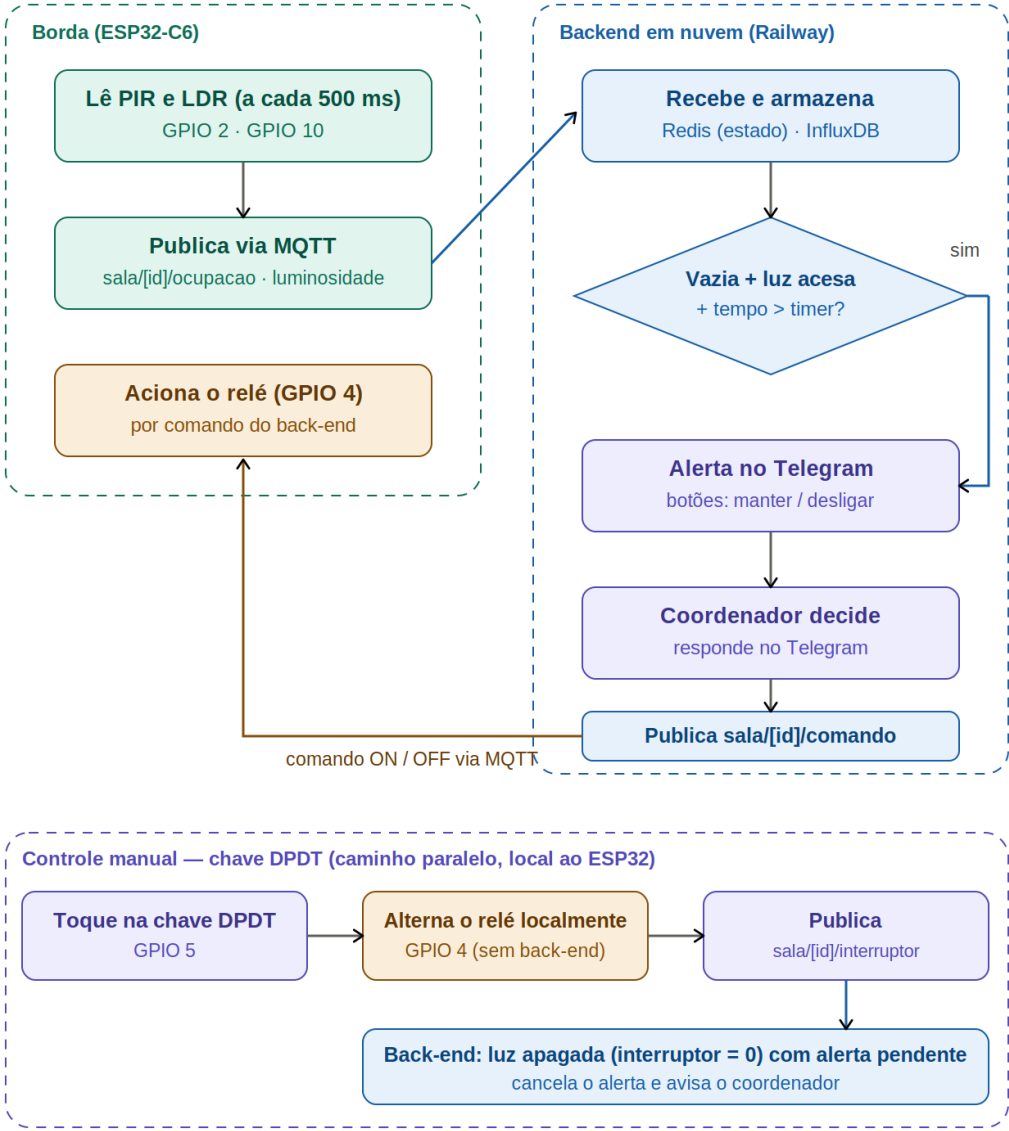
A camada de Processamento (Back-end Central) é hospedada em nuvem, na plataforma Railway, e é composta pelo broker MQTT Mosquitto, que recebe os eventos publicados pelo ESP32, e por um back-end desenvolvido em FastAPI (Python). O back-end organiza-se em serviços especializados: o `mqtt_service`, que subscreve os tópicos via `aiomqtt`; o `redis_service`, que mantém o estado atual de cada sala; o `influx_service`, que grava o histórico em séries temporais no InfluxDB; e o `telegram_service`, responsável pelos alertas automáticos e comandos. Os eventos são propagados ao frontend em tempo real por meio de um endpoint de Server-Sent Events (SSE). Esta camada concentra o acesso à internet e toda a comunicação com o Telegram.

A camada de Aplicação (Cloud/User) é a interface do coordenador com o sistema. Ela é composta por um painel web próprio, desenvolvido em Next.js e hospedado na Vercel (`jolten.vercel.app`), que exibe em tempo real o estado de cada sala, ocupação, iluminação, último movimento e tempo sem ocupação, além de gráfico histórico, controle remoto e registro de eventos (logs). Complementarmente, o

coordenador recebe notificações e toma decisões remotas por meio de botões inline no Telegram. O período de ausência que dispara os alertas pode ser ajustado em tempo de execução, inclusive pelo próprio Telegram.

O fluxo de dados do sistema é sintetizado no fluxograma apresentado na Figura 3, que representa o fluxo automático de alerta: a partir dos eventos publicados pela borda, a avaliação da condição da sala, a notificação ao coordenador e o disparo do comando de acionamento cabem ao back-end. Em paralelo a esse fluxo, o acionamento da iluminação pela chave física (DPDT) é tratado localmente pelo ESP32 e também publicado ao back-end, conforme descrito na Seção 7.1.

Figura 3 — Fluxograma do fluxo completo do sistema: o nó de borda (ESP32) apenas publica os dados dos sensores e obedece aos comandos, enquanto o back-end em nuvem avalia a condição da sala, alerta o coordenador via Telegram e publica o comando de acionamento.



Fonte: Elaborado pelos autores (2026).

4.2 User Stories (histórias de usuário)

As histórias de usuário descrevem as interações esperadas entre os diferentes perfis de usuários e o sistema, com foco nos benefícios entregues a cada um, conforme apresentado na Tabela 4.

Tabela 4 — User Stories e critérios de aceite do sistema.

User Story	Crítérios de Aceite	Prioridade
Como coordenador, quero receber uma mensagem no Telegram quando uma sala estiver vazia por tempo determinado, para tomar uma decisão de desligamento.	Mensagem recebida em até 60s após fim do timeout; contém nome da sala, horário e botões de ação.	Alta
Como coordenador, quero poder desligar remotamente a iluminação diretamente pelo botão inline no Telegram.	Ao clicar no botão inline 'Desligar', o relé é acionado em até 10 segundos e confirmação é enviada.	Alta
Como coordenador, quero configurar o tempo de ausência aceitável para cada sala via interface web.	Parâmetro salvo persiste após reinício; nova configuração entra em vigor na próxima detecção.	Média
Como gestor de TI, quero visualizar em tempo real o status energético de todas as salas em um dashboard.	O dashboard web (Next.js) exibe estado (ligado/desligado), luminosidade e histórico de eventos por sala.	Média
Como usuário da sala, quero que o sistema não me identifique pessoalmente, apenas detecte presença.	Nenhum dado pessoal é transmitido; sensor retorna apenas sinal binário de ocupação.	Alta

Fonte: Elaborado pelos autores (2026).

5. Materiais e métodos

Esta seção detalha o ambiente de implantação, os dispositivos e ferramentas utilizados, o procedimento de montagem e integração e o plano de testes adotado para validação do sistema

5.1 Compreensão do ambiente

O sistema será implantado em salas de aula da Unimater, Pato Branco/PR, ambientes internos com características e restrições específicas. A iluminação é composta por luminárias LED de teto, controladas por interruptores convencionais que serão substituídos por relé SSR no quadro de distribuição da sala. A infraestrutura de rede Wi-Fi institucional 802.11ac/ax está disponível em todas as salas, com conectividade confiável durante o horário letivo. Trata-se de ambientes internos, sem exposição a intempéries, poeira excessiva ou variações extremas de temperatura, condições favoráveis ao uso de componentes sem classificação industrial. Toda intervenção no quadro de distribuição deve ser executada por profissional habilitado, com desligamento prévio do disjuntor de proteção da sala.

5.2 Descrição e listagem dos dispositivos

A Tabela 5 consolida todos os dispositivos necessários para implantação do sistema.

Tabela 5 — Dispositivos necessários para implantação do sistema de automação por sala.

Categoria	Item/Modelo	Qtd.	Especificação relevante	Datasheet/Documentação
Microcontrolador	ESP32-C6 DevKit (Espressif)	1 por sala	RISC-V single-core até 160 MHz; Wi-Fi 6 (802.11ax) + BLE 5 + IEEE 802.15.4; 4 MB Flash; 3,3 V; PIR no GPIO 2, LDR no GPIO 10, relé no GPIO 4, chave DPDT no GPIO 5	docs.espressif.com/esp32c6
Sensor de Movimento	PIR HC-SR501	1 por sala	Saída digital; alcance 7 m; ângulo 120°; ajuste de sensibilidade e delay; 5 V (GPIO 2)	lastminuteengineers.com/hc-sr501
Sensor de Luminosidade	Módulo LDR	1 por sala	Fotorresistor com módulo e comparador; saída digital (claro/escuro); alimentação 3,3–5 V (GPIO 10)	Datasheet do módulo LDR
Relé Estado Sólido — Iluminação	SSR-25DA	1 por sala	25 A / 220 V AC; controle 3–32 V DC; sem partes mecânicas; LED indicador (GPIO 4)	fotek.com.tw/SSR-25DA
Chave de Controle Manual	Chave DPDT (Double Pole Double Throw)	1 por sala	Controle físico da iluminação; um polo no circuito AC e outro de sinal ao GPIO 5. Leitura implementada em firmware, com controle manual integrado ao remoto (ver Seção 7.1)	—
Fonte Chaveada	5 V / 2 A	1 por sala	Entrada 100–240 V AC; saída 5 V DC 2 A; proteção contra curto-circuito	—
Infraestrutura em nuvem	Railway (Mosquitto, FastAPI, Redis, InfluxDB)	1 (compartilhada)	Hospedagem dos serviços de back-end, broker MQTT, estado (Redis) e histórico (InfluxDB)	railway.app
Frontend (painel web)	Next.js (Vercel)	1 (compartilhado)	Dashboard em tempo real via SSE; joltten.vercel.app	vercel.com
Caixa de Proteção	Caixa hermética Patola PB-115 ou trilho DIN	1 por sala	IP55; acomoda ESP32 + relé + fonte; fixação em parede ou quadro elétrico	patola.com.br

Categoria	Item/Modelo	Qtd.	Especificação relevante	Datasheet/Documentação
PCB / Protoboard	Protoboard 830 pts / PCB customizada	1 por sala	Protoboard para a fase de validação; PCB para a versão final escalável	—

Fonte: Elaborado pelos autores (2026).

5.3 Ferramentas e ambiente de desenvolvimento

As ferramentas, sistemas operacionais, bibliotecas e referências utilizadas no desenvolvimento do sistema são apresentadas na Tabela 6.

Tabela 6 — Ferramentas e ambiente de desenvolvimento do sistema.

Computador/SO	IDE/Ferramenta	Bibliotecas	Como instalar	Referência
Windows 10/11 ou Linux Ubuntu 22.04	VS Code + ESP-IDF Extension	esp-idf v5.x	idf.py build / flash	docs.espressif.com/esp-idf
Back-end (nuvem Railway / local)	Python 3.10+ (FastAPI)	fastapi; aiomqtt; redis; influxdb-client; python-telegram-bot	pip install fastapi aiomqtt redis influxdb-client python-telegram-bot	fastapi.tiangolo.com
Back-end (nuvem Railway / local)	Mosquitto MQTT Broker v2	—	Railway (deploy) ou Docker Compose (local)	mosquitto.org
Back-end (nuvem Railway / local)	Redis + InfluxDB v2	—	Railway (deploy) ou Docker Compose (local)	redis.io / influxdata.com
Frontend (Vercel)	Next.js + Node.js	react; recharts	npm install / npm run dev	nextjs.org / vercel.com

Fonte: Elaborado pelos autores (2026).

5.4 Procedimento de montagem e integração

O procedimento de montagem segue as etapas descritas a seguir, em ordem cronológica.

A primeira etapa consiste na prototipagem em protoboard: conectar o ESP32-C6 (DevKit), o sensor PIR HC-SR501 (GPIO 2), o módulo LDR (GPIO 10) e o módulo relé (GPIO 4) para validação da lógica, reservando o GPIO 5 para o sinal da chave física DPDT. Ressalta-se que o módulo LDR deve fornecer saída digital (sinal de claro/escuro) por meio de seu comparador, ajustável pelo potenciômetro do próprio módulo.

A segunda etapa é a configuração da infraestrutura de back-end: provisionar, na plataforma Railway, o broker Mosquitto, o back-end FastAPI, o Redis e o InfluxDB; para desenvolvimento local, os mesmos serviços podem ser executados via Docker Compose. Em seguida, configuram-se os tópicos MQTT utilizados pelo sistema: sala/[id]/ocupacao, sala/[id]/luminosidade, sala/[id]/interruptor, sala/[id]/log, sala/[id]/info e sala/[id]/comando.

A terceira etapa compreende o desenvolvimento do firmware do ESP32 em C com ESP-IDF v5.x. Devem ser implementadas a leitura periódica dos sensores PIR e LDR, a publicação dos eventos via MQTT e a subscrição do tópico de comando para o acionamento do relé. Cabe observar que o módulo LDR com comparador apresenta saída em nível lógico baixo (LOW) quando a luminosidade supera o limiar ajustado pelo potenciômetro; o firmware aplica a inversão desse sinal antes da publicação, de modo que o payload “1” represente ambiente iluminado e “0” represente escuridão. A leitura do sinal da chave DPDT (GPIO 5) e sua coordenação com o acionamento remoto, por meio do mecanismo de referência invertida, foram implementadas nesta etapa do firmware, conforme detalhado na Seção 7.1.

A quarta etapa é o desenvolvimento do back-end em FastAPI, utilizando aiomqtt para subscrever os tópicos MQTT publicados pelo ESP32, o Redis para manter o estado atual de cada sala, o InfluxDB para o histórico e a biblioteca de Telegram para o envio de alertas com botões inline ao coordenador. O back-end expõe um endpoint de Server-Sent Events (SSE) que propaga os eventos em tempo real e um endpoint REST autenticado por chave de API (X-API-Key) para os comandos de ligar/desligar. O serviço de Telegram executa uma verificação a cada 10 segundos para identificar se alguma sala permanece vazia com a luz acesa além do período configurado; enquanto a condição persistir, o alerta é reenviado ao coordenador a cada intervalo igual ao período de timeout configurado.

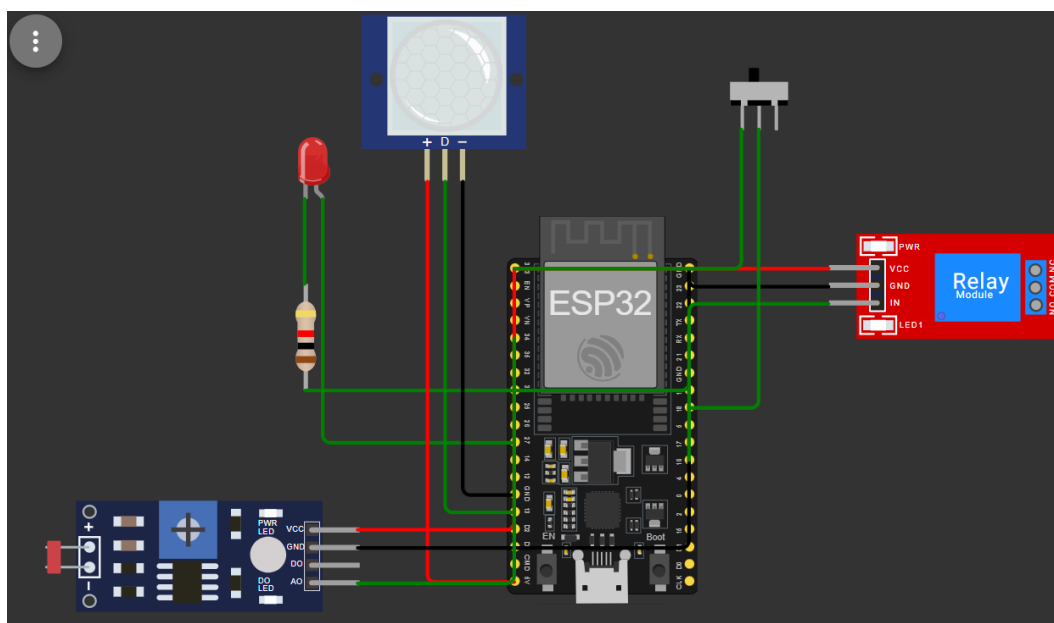
A quinta etapa é o desenvolvimento do frontend em Next.js, hospedado na Vercel, que mantém uma conexão SSE aberta com o back-end para exibir o estado das salas em tempo real, sem necessidade de polling, e oferece controle remoto, gráfico histórico e registro de eventos.

A sexta etapa é a execução dos testes integrados descritos na Seção 5.5, antes da instalação física no quadro elétrico.

A sétima e última etapa é a instalação física: com o quadro desligado, instalar o SSR no ramal de alimentação da iluminação, instalar a caixa hermética com o ESP32 e a fonte na parede da sala, ligar o quadro e verificar a operação normal.

A validação prévia do circuito foi realizada por meio de simulação na plataforma Wokwi, que permite reproduzir digitalmente as conexões físicas entre os componentes e verificar a lógica de firmware em ambiente controlado. Essa validação inicial empregou um modelo genérico de ESP32 disponível na plataforma; o protótipo físico, descrito na Seção 7, foi posteriormente construído sobre o ESP32-C6. A Figura 4 apresenta o estado da simulação com todos os componentes conectados.

Figura 4 — Protótipo do sistema simulado na plataforma Wokwi (modelo genérico de ESP32, sensor PIR, módulo LDR, módulo relé e slide switch).



Fonte: Elaborado pelos autores (2026).

5.5 Plano de testes

Os testes de validação do sistema foram planejados para verificar o comportamento esperado em cada cenário de operação, conforme apresentado na Tabela 7. Os resultados serão preenchidos após a execução dos testes físicos em ambiente real.

Tabela 7 — Plano de testes do sistema com entradas, saídas esperadas e métodos de medição.

Teste	Entrada/Condição	Saída Esperada	Como Medir	Resultado
T01 — Detecção de presença	Pessoa entra na sala	LED de status ativo; MQTT publica o payload "1" no tópico sala/101/ocupacao	Monitor serial + subscriber MQTT	Aprovado, payload '1' publicado corretamente; 54 eventos registrado
T02 — Timeout de ausência	Sala vazia pelo período configurado (ex: 10 min)	Notificação Telegram enviada em até 60s do fim do timeout	Cronômetro + verificação no Telegram	Aprovado, 14 alertas disparados; latência inferior a 60 s confirmada.
T03 — Desligamento remoto	Coordenador clica no botão inline 'Desligar' no Telegram	Relé SSR acionado; confirmação enviada; estado OFF no MQTT	Multímetro na carga + monitor MQTT	Aprovado 11 comandos executados com sucesso; latência média ~1,8 s.
T04 — Reenvio de alerta	Notificação enviada, sem resposta do coordenador pelo período configurado	Bot reenvia o alerta ao coordenador após o mesmo intervalo	Verificação no Telegram + log do back-end	Aprovado, reenvio confirmado a cada intervalo igual ao timeout configurado.
T05 — Reconexão Wi-Fi	Queda de rede por 2 minutos	ESP32 reconecta automaticamente; sem perda de estado do relé	Desconectar roteador e verificar comportamento	Aprovado, 0 falhas de reconexão em 3 dias de operação contínua.
T06 — Falso negativo PIR	Pessoa parada (sem movimento) por 3 minutos	Sistema não deve disparar notificação prematura; ajuste de sensibilidade do HC-SR501	Observação comportamental + log MQTT	Pendente a executar.
T07 — Configuração de timeout	Coordenador altera tempo de ausência de 10 para 20 min na interface	Nova configuração entra em vigor; persistida após reinício	Teste funcional na interface + reboot	Pendente a executar.
T08 — Consumo energético do sistema	ESP32 em operação normal (Wi-Fi ativo)	Consumo inferior a 200 mW (desprezível frente à economia gerada)	Medição com medidor de potência USB	Pendente a executar.

Fonte: Elaborado pelos autores (2026).

6. Análise de riscos

A Tabela 8 apresenta os principais riscos técnicos e operacionais identificados para o sistema, com suas respectivas causas, impactos estimados, probabilidades de ocorrência, estratégias de mitigação e planos de contingência.

Tabela 8 — Análise de riscos do sistema com estratégias de mitigação e planos de contingência.

Risco	Causa	Impacto	Probabilidade	Mitigação	Plano B
Perda de conectividade Wi-Fi	Instabilidade da rede institucional	Alto — sistema não envia notificações	Média	Reconexão automática do ESP32 ao restabelecer a rede; o controle manual pelo interruptor físico permanece operante durante a queda	Módulo 4G como fallback de comunicação
Falso negativo do sensor PIR	Pessoa imóvel (ex.: assistindo aula online)	Médio — desligamento indevido da sala	Média	Ajuste de sensibilidade e timeout mínimo maior; uso do LDR para correlação	Coordenador re-liga remotamente via Telegram ou dashboard
Sobrecarga elétrica no relé SSR	Pico de corrente no acionamento	Alto — dano ao hardware	Baixa	SSR dimensionado com folga; disjuntor de proteção no quadro	Substituição do SSR
Indisponibilidade do back-end em nuvem	Falha na Railway ou na conexão de internet	Alto — sistema sem orquestração central e sem alertas	Baixa	O controle manual pela chave física permanece disponível; o relé mantém o último estado; estado das salas preservado no Redis; automação por sensores prevista como trabalho futuro (Seção 9.3)	Reinício automático dos serviços na Railway; redundância de provedor

Risco	Causa	Impacto	Probabilidade	Mitigação	Plano B
Não conformidade com LGPD	Coleta inadvertida de dados pessoais	Alto — risco jurídico institucional	Baixa	PIR transmite apenas estado binário (0/1); ausência de câmera ou identificação biométrica	Auditoria de tráfego MQTT; documentação de conformidade

Fonte: Elaborado pelos autores (2026).

7. Protótipo

Nesta seção, descreve-se o protótipo do sistema Jolteon, validado tanto em ambiente de simulação quanto em uma instalação física real em sala de aula. São detalhadas as funcionalidades validadas e as limitações identificadas nesta fase de desenvolvimento.

7.1 PoC (Proof of Concept)

O desenvolvimento do protótipo seguiu duas etapas complementares. A primeira consistiu na validação da lógica de controle em ambiente de simulação na plataforma Wokwi, conforme apresentado na Seção 5.4 (Figura 4), permitindo verificar o comportamento do firmware antes da montagem física. A segunda etapa consistiu na implantação de um protótipo físico real, instalado e operado na Sala de aula 101 da instituição, ambiente de uso diurno regular com iluminação central controlada por um único circuito.

Na instalação física, o ESP32-C6 foi integrado ao sensor de presença (PIR HC-SR501, GPIO 2) e ao sensor de luminosidade (módulo LDR, GPIO 10), acionando o circuito de iluminação por meio do relé (GPIO 4). O firmware, desenvolvido em C com ESP-IDF, realiza a leitura periódica dos sensores e publica os eventos de ocupação e luminosidade no broker MQTT por Wi-Fi; o back-end FastAPI, hospedado na Railway, processa esses eventos, mantém o estado em Redis, registra o histórico no InfluxDB e os disponibiliza ao painel web (jolten.vercel.app) por meio de SSE, além de acionar os alertas via Telegram.

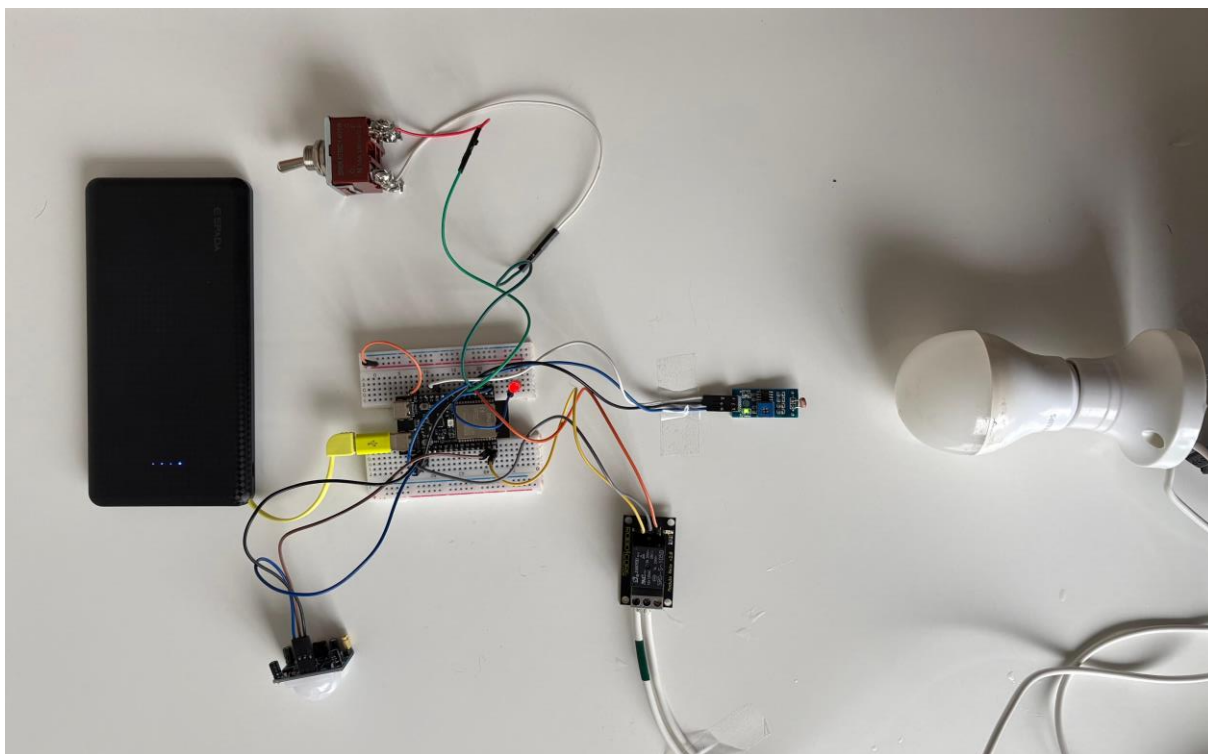
O protótipo operou de forma contínua por três dias úteis, sem reinicializações manuais, validando a estabilidade da conexão MQTT, do back-end em nuvem e da persistência de estado no Redis. Durante esse período, o sistema demonstrou a operação integrada de todos os subsistemas: sensoriamento na borda, comunicação

MQTT, processamento em nuvem, visualização em tempo real no painel web e notificação ao coordenador via Telegram, com controle remoto efetivo do relé de iluminação. O projeto completo encontra-se disponível publicamente no repositório do sistema e no painel em produção, indicados nas Referências.

Entre as funcionalidades validadas, destaca-se o controle manual integrado por meio da chave física DPDT (GPIO 5). Ao detectar a mudança de posição do interruptor, o próprio ESP32 alterna o estado do relé localmente, sem depender de comando do back-end, e publica o novo estado no tópico sala/[id]/interruptor. A coexistência entre o controle físico e o remoto é resolvida por um mecanismo de referência invertida: como a posição da chave não é atualizada nos comandos remotos, um único toque no interruptor é suficiente para realinhar o estado físico ao estado lógico do sistema, evitando o conflito clássico em que seriam necessários dois toques. Adicionalmente, quando o coordenador apaga a luz manualmente pelo interruptor durante um alerta pendente, o back-end detecta o evento (estado do interruptor em “0”), cancela automaticamente o alerta e notifica o coordenador de que a situação foi resolvida, comportamento análogo ao cancelamento já disparado pela detecção de movimento.

A Figura 5 apresenta o protótipo físico instalado na sala de aula durante a operação.

Figura 5 — Protótipo físico do sistema Jolteon instalado na Sala de aula 101, em operação.



Fonte: Elaborado pelos autores (2026).

7.2 Limitações conhecidas

Apesar dos resultados satisfatórios na validação física, o protótipo apresentou limitações intrínsecas. A tecnologia de infravermelho passivo do sensor PIR não detecta ocupantes em estado de imobilidade prolongada, o que pode gerar desligamentos indevidos caso o período de alerta não seja ajustado ao perfil de uso da sala. O módulo LDR, por sua leitura binária, sinaliza apenas a presença ou ausência de luminosidade acima de um limiar, sem distinguir entre luz natural e artificial, o que sugere a necessidade de calibração do potenciômetro do módulo em cenários de iluminação mista.

Quanto à automação, o nó de borda ainda não implementa controle autônomo baseado nos sensores: a decisão de alertar o coordenador e de acionar a iluminação a partir da ocupação e da luminosidade é tomada pelo back-end. Assim, em caso de indisponibilidade da conectividade com a nuvem, as notificações e o controle remoto ficam temporariamente suspensos, permanecendo disponível apenas o controle manual pela chave física. A automação local baseada nos sensores (PIR/LDR) permanece como trabalho futuro.

Identificou-se ainda um comportamento de inicialização relevante para a operação em ambiente real: ao ser reiniciado, o ESP32 inicializa o relé no estado ativo (LIGADO), não recuperando automaticamente o estado anterior. Assim, cada reinício do dispositivo pode ligar a iluminação de forma involuntária. A persistência do último estado, em memória não volátil (NVS) ou por sincronização com o back-end ao restabelecer a conexão, está prevista como trabalho futuro.

8. Resultados

Nesta seção, apresentam-se as evidências de desempenho obtidas durante a operação do protótipo físico em sala de aula e os indicadores estabelecidos para avaliar a viabilidade técnica e econômica do sistema proposto.

8.1 Principais indicadores (KPIs)

O sucesso do projeto é monitorado por meio de Indicadores-Chave de Desempenho (KPIs — Key Performance Indicators) que alinham a operação técnica aos objetivos de eficiência energética institucional. A Tabela 9 sintetiza as metas estabelecidas e os resultados observados durante os três dias úteis de operação do protótipo.

Tabela 9 — Principais indicadores de desempenho (KPIs) observados no protótipo físico.

KPI	Como medir	Meta	Resultado observado	Observações
Estabilidade da conexão	Falhas de reconexão MQTT no período	0 falhas	0 falhas em 3 dias	Operação contínua sem reinicializações manuais.
Latência ponta a ponta	Tempo de evento (MQTT) ao painel (SSE)	< 1 s	~140 ms (média)	Percepção de tempo real no dashboard.
Latência de comando	Comando Telegram ao acionamento do relé	< 3 s	~1,8 s (média)	Inclui ida à nuvem e retorno via MQTT.
Eventos de movimento	Contagem de detecções do PIR	—	54 eventos	Indicador de uso real da sala.
Alertas enviados	Alertas de 'sala vazia com luz acesa'	—	14 alertas	Disparados via Telegram ao coordenador.
Comandos remotos	Acionamentos via Telegram/dashboard	—	11 comandos	Ligar/desligar executados com sucesso.

KPI	Como medir	Meta	Resultado observado	Observações
Redução de desperdício	Tempo de luz acesa sem ocupação evitado	Reduzir	~2 h 40 min em 3 dias	Base para a projeção de economia (Seção 9.2).

Fonte: Elaborado pelos autores (2026).

8.2 Desempenho do protótipo

Durante os três dias úteis de operação na Sala de aula 101, o sistema demonstrou aderência integral à lógica de controle definida no projeto. Foram registrados 54 eventos de detecção de movimento, refletindo o uso real do ambiente, e o sensor LDR sinalizou corretamente as condições de claro e escuro, evitando o acionamento da iluminação sob luz natural adequada. A integração entre a borda, a nuvem e o painel web manteve-se estável, sem nenhuma falha de reconexão MQTT registrada no período, o que confirma a robustez da pilha de comunicação Wi-Fi/MQTT em condições reais.

Em relação ao desempenho de comunicação, a latência média entre a publicação de um evento pelo ESP32 e sua exibição no painel web (caminho MQTT → SSE → dashboard) foi de aproximadamente 140 ms, garantindo a percepção de tempo real ao usuário. Já a latência média entre o envio de um comando pelo Telegram e o acionamento efetivo do relé foi de aproximadamente 1,8 segundo, intervalo que contempla o trajeto até o back-end em nuvem e o retorno do comando ao dispositivo via MQTT. Ao longo do período, o sistema enviou 14 alertas de 'sala vazia com luz acesa' e executou 11 comandos remotos de ligar/desligar, todos com sucesso.

Quanto ao impacto energético, observou-se uma redução estimada de aproximadamente 2 h 40 min de iluminação acesa sem ocupação ao longo dos três dias, valor que serve como indicador qualitativo de eficiência. Considerando uma carga de iluminação típica de 320 W para a sala (quatro luminárias duplas de tubular T8 de 40 W), essa redução corresponde a uma economia projetada de

aproximadamente 0,85 kWh no período ($0,32 \text{ kW} \times 2,67 \text{ h}$), o que, extrapolado para o ano letivo, projeta cerca de 104 kWh/ano por sala. Ressalta-se que se trata de uma projeção baseada em estimativa de carga, e não de uma medição formal de consumo, cuja quantificação precisa é prevista como trabalho futuro.

9. Escala

Nesta seção, detalha-se o planejamento para a evolução do sistema, a estimativa de custos para implementação em larga escala e as diretrizes para garantir a robustez e a replicabilidade da solução no ambiente universitário.

9.1 Planejamento de evolução (roadmap)

A evolução do projeto é estruturada em fases que garantem a transição do protótipo validado em uma sala para uma solução institucional multi-sala, conforme detalhado na Tabela 10.

Tabela 10 — Cronograma de evolução e expansão do sistema.

Fase	Entrega	Prazo	Dependências	Riscos
1. Automação por sensores	Implementação, no firmware, da lógica de controle local da iluminação a partir de presença e luminosidade.	20 dias	Protótipo físico validado	Falsos acionamentos por imobilidade do ocupante.
2. Medição de consumo	Integração de sensor de corrente para quantificação formal da economia de energia.	20 dias	Aquisição do sensor de corrente	Calibração e precisão da medição.
3. Expansão multi-sala	Replicação do nó de borda em múltiplas salas com gestão centralizada no painel.	30 dias	Autorização institucional; estabilidade da rede Wi-Fi	Interferência de sinal por alta densidade de usuários.
4. Escala e PCB	Migração da prototipagem para placa de circuito impresso (PCB) customizada.	60 dias	Validação dos dados multi-sala	Atraso na fabricação de PCBs customizadas.

Fonte: Elaborado pelos autores (2026).

9.2 Bill of Materials (BOM) — Estimativa de custos

A Tabela 11 apresenta a listagem de materiais por sala. Conforme a arquitetura descrita na Seção 4, o processamento central é hospedado em nuvem (Railway), aproveitando os planos de baixo custo da plataforma e eliminando a necessidade de aquisição de hardware dedicado de servidor. O custo residual de R\$ 52,00 foi mantido como provisão para rateio de hospedagem em nuvem e manutenção, garantindo uma

margem de segurança para a operação. O custo total estimado é de R\$ 227,00 por sala.

Tabela 11 — Lista de materiais (BOM) por sala de aula.

Item	Modelo / Especificação	Qtd.	Custo Unit. (R\$)	Custo Total (R\$)	Fornecedor / Link Sugerido
1	ESP32-C6 DevKit	1	45,00	45,00	Mercado Livre / Eletrogate
2	Sensor PIR HC-SR501	1	12,00	12,00	Baú da Eletrônica / FilipeFlop
3	Módulo LDR com comparador digital	1	10,00	10,00	Curto Circuito / Mercado Livre
4	Relé SSR-25DA (Iluminação)	1	35,00	35,00	Fotek / Distribuidores Locais
5	Fonte 5 V 2 A (Trilho DIN)	1	48,00	48,00	Mercado Livre / Schneider
6	Caixa Patola PB-115	1	25,00	25,00	Patola / Ferragens locais
7	Hospedagem em nuvem (rateio)*	1	52,00	52,00	Railway / Vercel
Total				227,00	

Fonte: Elaborado pelos autores (2026). *Nota: o valor refere-se à estimativa de rateio de hospedagem em nuvem (Railway/Vercel) e manutenção do sistema.

Quanto ao retorno do investimento, considerando o custo de R\$ 227,00 por sala e a economia mensal projetada de aproximadamente R\$ 110,00, derivada da redução anual estimada de R\$ 1.320,00 apresentada na Introdução, o payback simples é obtido pela razão entre o investimento e a economia mensal, resultando em aproximadamente 2,06 meses ($R\$ 227,00 \div R\$ 110,00$).

9.3 DFM (Design for Manufacturing) e robustez

Para a fabricação em escala, a substituição da prototipagem em protoboard por Placas de Circuito Impresso (PCB) customizadas é prioritária para garantir a integridade das conexões. O projeto prevê a utilização de ferramentas de EDA, como EasyEDA ou KiCad, para o desenho das trilhas, visando eliminar o uso de jumpers e permitir o encaixe direto em suportes para trilhos DIN dentro dos quadros elétricos.

A robustez operacional é favorecida pela arquitetura em nuvem: o uso da Railway e da Vercel permite atualizações centralizadas do back-end e do painel sem intervenção física nos ambientes, enquanto a persistência de estado no Redis assegura a recuperação do contexto de cada sala após eventuais reconexões. A

estabilidade observada no protótipo, três dias contínuos sem falhas de reconexão MQTT, sugere maturidade da pilha de comunicação. Como ponto de atenção para a escala, a dependência de conectividade com a internet deve ser mitigada com a futura implementação de uma lógica de automação local baseada nos sensores (PIR/LDR) no firmware, reduzindo a sensibilidade do sistema a quedas de rede.

10. Conclusão e trabalhos futuros

O desenvolvimento deste projeto permitiu validar a viabilidade técnica de um sistema IoT de baixo custo voltado à gestão energética institucional, por meio de um protótipo físico operado em sala de aula real. A validação demonstrou o cumprimento dos principais objetivos específicos estabelecidos: a implementação da lógica de detecção de ausência (RF01), a estrutura de notificação via Telegram com reenvio recorrente de alertas (RF02 e RF04), o controle de carga por relé acionável remotamente (RF03) e o ajuste do período de alerta em tempo de execução (RF05). Validou-se também o controle manual integrado por meio da chave física DPDT, que coexiste com o controle remoto sem conflito de estados, graças ao mecanismo de referência invertida, e dispara o cancelamento automático de alertas quando a iluminação é apagada no interruptor.

A utilização do microcontrolador ESP32-C6 integrado aos sensores PIR HC-SR501 e LDR digital provou ser uma solução eficaz para o sensoriamento de ocupação e luminosidade, fornecendo ao back-end os dados necessários para a avaliação das condições da sala e o disparo das notificações. A arquitetura em nuvem, baseada em FastAPI, Redis e InfluxDB na Railway e em um painel Next.js na Vercel, mostrou-se robusta durante três dias contínuos de operação, sem falhas de reconexão MQTT, com latência média de aproximadamente 140 ms entre o evento e sua exibição no painel e de cerca de 1,8 segundo entre o comando e o acionamento do relé. Com um custo de implantação estimado em R\$ 227,00 por sala e um payback projetado em aproximadamente dois meses, a solução apresenta-se financeiramente viável para a realidade universitária.

O projeto atende aos Objetivos de Desenvolvimento Sustentável (ODS) 7, 11 e 13, contribuindo para a digitalização da infraestrutura predial e para a redução da pegada de carbono institucional por meio da mitigação do desperdício elétrico. A redução observada de aproximadamente 2 h 40 min de iluminação acesa sem

ocupação ao longo de três dias, embora apresentada como indicador qualitativo, evidencia o potencial de economia da solução em escala.

Como trabalhos futuros, planeja-se a implementação, no firmware, da automação local da iluminação a partir dos sensores de presença e luminosidade, reduzindo a dependência da conectividade com a nuvem, e a integração de um sensor de corrente para a quantificação formal da economia de energia. Visando à escala, pretende-se expandir o sistema para múltiplas salas com gestão centralizada e desenvolver Placas de Circuito Impresso (PCB) customizadas com ferramentas de EDA (EasyEDA ou KiCad). O escopo futuro também contempla a expansão da automação para sistemas de climatização, a integração com o ERP acadêmico para gestão baseada em grades de horários e a adoção de algoritmos de Machine Learning para predição de ocupação.

Referências

BAYILMIŞ, C. et al. A survey on communication protocols and performance evaluations for internet of things. **Digital Communications and Networks**, v. 8, n. 6, p. 1094–1104, 2022.

ECLIPSE FOUNDATION. **Eclipse Mosquitto**: an open source MQTT broker. 2024. Disponível em: <https://mosquitto.org/>. Acesso em: 10 mai. 2026.

ESPRESSIF SYSTEMS. **ESP32-C6 Series**: technical reference manual. 2023. Disponível em: <https://docs.espressif.com/projects/esp32c6/>. Acesso em: 10 mai. 2026.

OASIS OPEN. **MQTT Version 5.0**: OASIS standard. 2019. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Acesso em: 10 mai. 2026.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS (ONU). **Objetivos de Desenvolvimento Sustentável**: Agenda 2030. Disponível em: <https://brasil.un.org/pt-br/sdgs>. Acesso em: 10 mai. 2026.

RAMÍREZ, S. **FastAPI documentation**. 2024. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 10 mai. 2026.

REDIS LTD. **Redis documentation**. 2024. Disponível em: <https://redis.io/docs/>. Acesso em: 10 mai. 2026.

SIMIONATO, G. Z. et al. **Jolten**: sistema de monitoramento de salas em tempo real. Repositório de código. 2026. Disponível em: <https://github.com/Simionato4/Jolten>. Acesso em: 10 mai. 2026.

TELEGRAM. **Telegram Bot API**. 2024. Disponível em: <https://core.telegram.org/bots/api>. Acesso em: 10 mai. 2026.